NUTRIA — Onboarding Completo (13 archivos)

Todos los archivos del flujo de onboarding listos para copiar al proyecto.

---

src/app/(onboarding)/_layout.tsx

```
// src/app/(onboarding)/_layout.tsx
// Layout del grupo de onboarding: Stack con transición slide_from_right.
// Incluye barra de progreso animada visible en todos los pasos excepto welcome.

import { Stack } from "expo-router";
import { View } from "react-native";
import { useOnboardingStore } from "@/stores/onboardingStore";
import { StepProgressBar } from "@/components/onboarding/StepProgressBar";

export default function OnboardingLayout() {
  const currentStep = useOnboardingStore((s) => s.currentStep);

  return (
    <View className="flex-1 bg-neutral-50">
      {/* Barra de progreso: visible en pasos 2-7, oculta en welcome (paso 1) */}
      {currentStep > 1 && (
        <StepProgressBar currentStep={currentStep} totalSteps={7} />
      )}

      <Stack
        screenOptions={{
          headerShown: false,
          animation: "slide_from_right",
          gestureEnabled: false,
          contentStyle: { backgroundColor: "#FAFAF9" },
        }}
      >
        <Stack.Screen name="welcome" />
        <Stack.Screen name="body-profile" />
        <Stack.Screen name="goal" />
        <Stack.Screen name="activity" />
        <Stack.Screen name="tca-screening" />
        <Stack.Screen name="preferences" />
        <Stack.Screen name="ready" />
```

```
      </Stack>
    </View>
  );
}
```

---

src/app/(onboarding)/activity.tsx

```
// src/app/(onboarding)/activity.tsx
// Nivel de actividad — paso 4 del onboarding.

import { View, Text, Pressable, ScrollView } from "react-native";
import { useRouter } from "expo-router";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { cn } from "@/lib/cn";
import { useOnboardingStore } from "@/stores/onboardingStore";
import { ActivitySelector } from "@/components/onboarding/ActivitySelector";

export default function ActivityScreen() {
  const router = useRouter();
  const insets = useSafeAreaInsets();
  const { activityLevel, setField, nextStep, prevStep } = useOnboardingStore();

  const handleNext = () => {
    if (!activityLevel) return;
    nextStep();
    router.push("/(onboarding)/tca-screening");
  };

  const handleBack = () => {
    prevStep();
    router.back();
  };

  return (
    <View
      className="flex-1 bg-neutral-50"
      style={{ paddingBottom: insets.bottom + 24 }}
    >
      <ScrollView
        className="flex-1 px-6"
        contentContainerClassName="pt-4 pb-4"
        showsVerticalScrollIndicator={false}
      >
        <Text className="font-display text-2xl text-neutral-900 mb-2">
```

```
          ¿Cuánto te mueves?
        </Text>
        <Text className="text-neutral-500 mb-6">
          Elige la opción que mejor describa tu semana típica.
        </Text>
        <ActivitySelector
          value={activityLevel}
          onChange={(l) => setField("activityLevel", l)}
        />
      </ScrollView>

      <View className="flex-row gap-3 px-6">
        <Pressable
          onPress={handleBack}
          accessibilityLabel="Volver al paso anterior"
          accessibilityRole="button"
          className="flex-1 py-4 rounded-2xl items-center border border-neutral-2
        >
          <Text className="text-neutral-600 font-semibold text-base">Atrás</Text>
        </Pressable>
        <Pressable
          onPress={handleNext}
          disabled={!activityLevel}
          accessibilityLabel="Continuar al siguiente paso"
          accessibilityRole="button"
          className={cn(
            "flex-[2] py-4 rounded-2xl items-center",
            activityLevel
              ? "bg-primary-500 active:bg-primary-600"
              : "bg-neutral-200"
          )}
        >
          <Text
            className={cn(
              "font-semibold text-base",
              activityLevel ? "text-white" : "text-neutral-400"
            )}
          >
            Continuar
          </Text>
        </Pressable>
      </View>
    </View>
  );
}
```

src/app/(onboarding)/body-profile.tsx

```
// src/app/(onboarding)/body-profile.tsx
// Datos físicos del usuario — paso 2 del onboarding.
// Campos: nombre (opcional), peso, altura, fecha de nacimiento, sexo biológico.

import { useState } from "react";
import { View, Text, Pressable, TextInput, ScrollView, Alert } from "react-native
import { useRouter } from "expo-router";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from "react-native-reanimated";
import { cn } from "@/lib/cn";
import { useOnboardingStore } from "@/stores/onboardingStore";
import type { BiologicalSex } from "@/stores/onboardingStore";

export default function BodyProfileScreen() {
  const router = useRouter();
  const insets = useSafeAreaInsets();
  const {
    displayName,
    weightKg,
    heightCm,
    dateOfBirth,
    biologicalSex,
    unitWeight,
    setField,
    nextStep,
    prevStep,
  } = useOnboardingStore();

  // Estado local para inputs de texto (se sincronizan con store al cambiar)
  const [weightInput, setWeightInput] = useState(
    weightKg ? String(weightKg) : ""
  );
  const [heightInput, setHeightInput] = useState(
    heightCm ? String(heightCm) : ""
  );
  const [dobDay, setDobDay] = useState(
    dateOfBirth ? dateOfBirth.split("-")[2] : ""
  );
  const [dobMonth, setDobMonth] = useState(
    dateOfBirth ? dateOfBirth.split("-")[1] : ""
```

```
);
const [dobYear, setDobYear] = useState(
  dateOfBirth ? dateOfBirth.split("-")[0] : ""
);

// Validación
const validate = (): boolean => {
  const w = parseFloat(weightInput);
  const h = parseFloat(heightInput);
  const y = parseInt(dobYear, 10);
  const m = parseInt(dobMonth, 10);
  const d = parseInt(dobDay, 10);

  if (!w || w < 30 || w > 300) {
    Alert.alert("Peso", "Introduce un peso entre 30 y 300 kg");
    return false;
  }
  if (!h || h < 100 || h > 250) {
    Alert.alert("Altura", "Introduce una altura entre 100 y 250 cm");
    return false;
  }
  if (!y || !m || !d || y < 1920 || y > 2013 || m < 1 || m > 12 || d < 1 || d >
    Alert.alert(
      "Fecha de nacimiento",
      "Introduce una fecha válida (debes tener entre 13 y 100 años)"
    );
    return false;
  }
  if (!biologicalSex) {
    Alert.alert("Sexo biológico", "Selecciona una opción para calcular tu metab
    return false;
  }

  return true;
};

const handleNext = () => {
  if (!validate()) return;

  // Sincronizar con store
  const w = parseFloat(weightInput);
  const h = parseFloat(heightInput);
  const dob = `${dobYear}-${dobMonth.padStart(2, "0")}-${dobDay.padStart(2, "0"

  // Convertir si el usuario usa lb
  const weightInKg = unitWeight === "lb" ? w * 0.453592 : w;
```

```
  setField("weightKg", Math.round(weightInKg * 10) / 10);
  setField("heightCm", h);
  setField("dateOfBirth", dob);

  nextStep();
  router.push("/(onboarding)/goal");
};

const handleBack = () => {
  prevStep();
  router.back();
};

return (
  <ScrollView
    className="flex-1 bg-neutral-50"
    contentContainerClassName="px-6 pb-10"
    style={{ paddingTop: 16 }}
    keyboardShouldPersistTaps="handled"
  >
    {/* Header */}
    <Text className="font-display text-2xl text-neutral-900 mb-2">
      Cuéntanos sobre ti
    </Text>
    <Text className="text-neutral-500 mb-8">
      Necesitamos estos datos para calcular tu plan personalizado.
    </Text>

    {/* Nombre (opcional) */}
    <View className="mb-5">
      <Text className="text-sm font-medium text-neutral-700 mb-1.5">
        ¿Cómo te llamamos?{" "}
        <Text className="text-neutral-400">(opcional)</Text>
      </Text>
      <TextInput
        value={displayName}
        onChangeText={(t) => setField("displayName", t)}
        placeholder="Tu nombre"
        placeholderTextColor="#A8A29E"
        className="bg-white border border-neutral-200 rounded-xl px-4 py-3.5 te
        accessibilityLabel="Nombre para mostrar"
        autoCapitalize="words"
        autoCorrect={false}
      />
    </View>

    {/* Peso */}
```

```
<View className="mb-5">
  <Text className="text-sm font-medium text-neutral-700 mb-1.5">
    Peso actual ({unitWeight})
  </Text>
  <TextInput
    value={weightInput}
    onChangeText={setWeightInput}
    placeholder={unitWeight === "kg" ? "70" : "154"}
    placeholderTextColor="#A8A29E"
    keyboardType="decimal-pad"
    className="bg-white border border-neutral-200 rounded-xl px-4 py-3.5 te
    accessibilityLabel={`Peso actual en ${unitWeight}`}
  />
</View>

{/* Altura */}
<View className="mb-5">
  <Text className="text-sm font-medium text-neutral-700 mb-1.5">
    Altura (cm)
  </Text>
  <TextInput
    value={heightInput}
    onChangeText={setHeightInput}
    placeholder="170"
    placeholderTextColor="#A8A29E"
    keyboardType="number-pad"
    className="bg-white border border-neutral-200 rounded-xl px-4 py-3.5 te
    accessibilityLabel="Altura en centímetros"
  />
</View>

{/* Fecha de nacimiento — 3 inputs */}
<View className="mb-5">
  <Text className="text-sm font-medium text-neutral-700 mb-1.5">
    Fecha de nacimiento
  </Text>
  <View className="flex-row gap-3">
    <TextInput
      value={dobDay}
      onChangeText={(t) => setDobDay(t.replace(/[^0-9]/g, "").slice(0, 2))}
      placeholder="DD"
      placeholderTextColor="#A8A29E"
      keyboardType="number-pad"
      maxLength={2}
      className="flex-1 bg-white border border-neutral-200 rounded-xl px-4
      accessibilityLabel="Día de nacimiento"
    />
```

```
    <TextInput
      value={dobMonth}
      onChangeText={(t) => setDobMonth(t.replace(/[^0-9]/g, "").slice(0, 2)
      placeholder="MM"
      placeholderTextColor="#A8A29E"
      keyboardType="number-pad"
      maxLength={2}
      className="flex-1 bg-white border border-neutral-200 rounded-xl px-4
      accessibilityLabel="Mes de nacimiento"
    />
    <TextInput
      value={dobYear}
      onChangeText={(t) => setDobYear(t.replace(/[^0-9]/g, "").slice(0, 4))
      placeholder="AAAA"
      placeholderTextColor="#A8A29E"
      keyboardType="number-pad"
      maxLength={4}
      className="flex-[1.5] bg-white border border-neutral-200 rounded-xl p
      accessibilityLabel="Año de nacimiento"
    />
  </View>
</View>

{/* Sexo biológico — 2 cards */}
<View className="mb-8">
  <Text className="text-sm font-medium text-neutral-700 mb-1.5">
    Sexo biológico{" "}
    <Text className="text-neutral-400">(para calcular tu metabolismo)</Text
  </Text>
  <View className="flex-row gap-3">
    <SexCard
      label="Hombre"
      emoji="♂"
      isSelected={biologicalSex === "male"}
      onPress={() => setField("biologicalSex", "male")}
    />
    <SexCard
      label="Mujer"
      emoji="♀"
      isSelected={biologicalSex === "female"}
      onPress={() => setField("biologicalSex", "female")}
    />
  </View>
</View>

{/* Botones */}
<View className="flex-row gap-3">
```

```
        <Pressable
          onPress={handleBack}
          accessibilityLabel="Volver al paso anterior"
          accessibilityRole="button"
          className="flex-1 py-4 rounded-2xl items-center border border-neutral-2
        >
          <Text className="text-neutral-600 font-semibold text-base">
            Atrás
          </Text>
        </Pressable>
        <Pressable
          onPress={handleNext}
          accessibilityLabel="Continuar al siguiente paso"
          accessibilityRole="button"
          className="flex-[2] py-4 rounded-2xl items-center bg-primary-500 active
        >
          <Text className="text-white font-semibold text-base">Continuar</Text>
        </Pressable>
      </View>
    </ScrollView>
  );
}

// Subcomponente: card de selección de sexo biológico
function SexCard({
  label,
  emoji,
  isSelected,
  onPress,
}: {
  label: string;
  emoji: string;
  isSelected: boolean;
  onPress: () => void;
}) {
  const scale = useSharedValue(1);

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  const handlePress = () => {
    scale.value = withSpring(0.94, { damping: 15, stiffness: 300 }, () => {
      scale.value = withSpring(1, { damping: 12, stiffness: 200 });
    });
    onPress();
  };
```

```
  return (
    <Animated.View style={animatedStyle} className="flex-1">
      <Pressable
        onPress={handlePress}
        accessibilityLabel={`Sexo biológico: ${label}`}
        accessibilityRole="button"
        accessibilityState={{ selected: isSelected }}
        className={cn(
          "items-center py-5 rounded-xl border-2",
          isSelected
            ? "border-primary-500 bg-primary-50"
            : "border-neutral-200 bg-white"
        )}
      >
        <Text className="text-2xl mb-1">{emoji}</Text>
        <Text
          className={cn(
            "text-base font-medium",
            isSelected ? "text-primary-700" : "text-neutral-700"
          )}
        >
          {label}
        </Text>
      </Pressable>
    </Animated.View>
  );
}
```

---

src/app/(onboarding)/goal.tsx

```
// src/app/(onboarding)/goal.tsx
// Selección de objetivo — paso 3 del onboarding.

import { View, Text, Pressable } from "react-native";
import { useRouter } from "expo-router";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { cn } from "@/lib/cn";
import { useOnboardingStore } from "@/stores/onboardingStore";
import { GoalSelector } from "@/components/onboarding/GoalSelector";

export default function GoalScreen() {
  const router = useRouter();
  const insets = useSafeAreaInsets();
```

```
const { goal, setField, nextStep, prevStep } = useOnboardingStore();

const handleNext = () => {
  if (!goal) return;
  nextStep();
  router.push("/(onboarding)/activity");
};

const handleBack = () => {
  prevStep();
  router.back();
};

return (
  <View
    className="flex-1 bg-neutral-50 px-6 justify-between"
    style={{ paddingTop: 16, paddingBottom: insets.bottom + 24 }}
  >
    <View>
      <Text className="font-display text-2xl text-neutral-900 mb-2">
        ¿Cuál es tu objetivo?
      </Text>
      <Text className="text-neutral-500 mb-8">
        Ajustaremos tu plan según lo que quieres conseguir.
      </Text>
      <GoalSelector value={goal} onChange={(g) => setField("goal", g)} />
    </View>

    <View className="flex-row gap-3">
      <Pressable
        onPress={handleBack}
        accessibilityLabel="Volver al paso anterior"
        accessibilityRole="button"
        className="flex-1 py-4 rounded-2xl items-center border border-neutral-2
      >
        <Text className="text-neutral-600 font-semibold text-base">Atrás</Text>
      </Pressable>
      <Pressable
        onPress={handleNext}
        disabled={!goal}
        accessibilityLabel="Continuar al siguiente paso"
        accessibilityRole="button"
        className={cn(
          "flex-[2] py-4 rounded-2xl items-center",
          goal ? "bg-primary-500 active:bg-primary-600" : "bg-neutral-200"
        )}
      >
```

```
          <Text
            className={cn(
              "font-semibold text-base",
              goal ? "text-white" : "text-neutral-400"
            )}
          >
            Continuar
          </Text>
        </Pressable>
      </View>
    </View>
  );
}
```

---

src/app/(onboarding)/preferences.tsx

```
// src/app/(onboarding)/preferences.tsx
// Preferencias del usuario — paso 6 del onboarding.
// País (prioridad hispanos), unidades de peso y energía, timezone.

import { useState } from "react";
import {
  View,
  Text,
  Pressable,
  ScrollView,
  TextInput,
  FlatList,
} from "react-native";
import { useRouter } from "expo-router";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { cn } from "@/lib/cn";
import { useOnboardingStore } from "@/stores/onboardingStore";

// Países hispanos primero, luego otros
const COUNTRIES = [
  { code: "ES", name: "España", flag: "🇪🇸" },
  { code: "MX", name: "México", flag: "🇲🇽" },
  { code: "AR", name: "Argentina", flag: "🇦🇷" },
  { code: "CO", name: "Colombia", flag: "🇨🇴" },
  { code: "CL", name: "Chile", flag: "🇨🇱" },
  { code: "PE", name: "Perú", flag: "🇵🇪" },
  { code: "VE", name: "Venezuela", flag: "🇻🇪" },
  { code: "EC", name: "Ecuador", flag: "🇪🇨" },
```

```
  { code: "BO", name: "Bolivia", flag: "🇧🇴" },
  { code: "UY", name: "Uruguay", flag: "🇺🇾" },
  { code: "PY", name: "Paraguay", flag: "🇵🇾" },
  { code: "PA", name: "Panamá", flag: "🇵🇦" },
  { code: "CR", name: "Costa Rica", flag: "🇨🇷" },
  { code: "DO", name: "República Dominicana", flag: "🇩🇴" },
  { code: "GT", name: "Guatemala", flag: "🇬🇹" },
  { code: "HN", name: "Honduras", flag: "🇭🇳" },
  { code: "SV", name: "El Salvador", flag: "🇸🇻" },
  { code: "NI", name: "Nicaragua", flag: "🇳🇮" },
  { code: "CU", name: "Cuba", flag: "🇨🇺" },
  { code: "PR", name: "Puerto Rico", flag: "🇵🇷" },
  { code: "US", name: "Estados Unidos", flag: "🇺🇸" },
];

export default function PreferencesScreen() {
  const router = useRouter();
  const insets = useSafeAreaInsets();
  const {
    countryCode,
    unitWeight,
    unitEnergy,
    timezone,
    setField,
    nextStep,
    prevStep,
  } = useOnboardingStore();

  const [countrySearch, setCountrySearch] = useState("");
  const [showCountryList, setShowCountryList] = useState(false);

  const filteredCountries = countrySearch
    ? COUNTRIES.filter((c) =>
        c.name.toLowerCase().includes(countrySearch.toLowerCase())
      )
    : COUNTRIES;

  const selectedCountry = COUNTRIES.find((c) => c.code === countryCode);

  const handleSelectCountry = (code: string) => {
    setField("countryCode", code);
    setShowCountryList(false);
    setCountrySearch("");
  };

  const handleNext = () => {
    nextStep();
```

```
  router.push("/(onboarding)/ready");
};

const handleBack = () => {
  prevStep();
  router.back();
};

return (
  <View
    className="flex-1 bg-neutral-50"
    style={{ paddingBottom: insets.bottom + 24 }}
  >
    <ScrollView
      className="flex-1 px-6"
      contentContainerClassName="pt-4 pb-4"
      keyboardShouldPersistTaps="handled"
      showsVerticalScrollIndicator={false}
    >
      <Text className="font-display text-2xl text-neutral-900 mb-2">
        Tus preferencias
      </Text>
      <Text className="text-neutral-500 mb-8">
        Adaptamos la app a tu país y unidades preferidas.
      </Text>

      {/* País */}
      <View className="mb-6">
        <Text className="text-sm font-medium text-neutral-700 mb-1.5">País</Tex

        {/* Selector de país */}
        <Pressable
          onPress={() => setShowCountryList(!showCountryList)}
          accessibilityLabel={`País seleccionado: ${selectedCountry?.name || "N
          accessibilityRole="button"
          className="bg-white border border-neutral-200 rounded-xl px-4 py-3.5
        >
          <Text className="text-xl mr-3">{selectedCountry?.flag || "🌍"}</Text>
          <Text className="text-base text-neutral-900 flex-1">
            {selectedCountry?.name || "Selecciona tu país"}
          </Text>
          <Text className="text-neutral-400">
            {showCountryList ? "▲" : "▼"}
          </Text>
        </Pressable>

        {/* Lista desplegable de países */}
```

```
  {showCountryList && (
    <View className="bg-white border border-neutral-200 rounded-xl mt-2 m
      <TextInput
        value={countrySearch}
        onChangeText={setCountrySearch}
        placeholder="Buscar país..."
        placeholderTextColor="#A8A29E"
        className="px-4 py-3 border-b border-neutral-100 text-base text-n
        autoFocus
        accessibilityLabel="Buscar país"
      />
      <FlatList
        data={filteredCountries}
        keyExtractor={(item) => item.code}
        renderItem={({ item }) => (
          <Pressable
            onPress={() => handleSelectCountry(item.code)}
            className={cn(
              "flex-row items-center px-4 py-3 border-b border-neutral-50
              countryCode === item.code && "bg-primary-50"
            )}
            accessibilityLabel={item.name}
            accessibilityRole="button"
          >
            <Text className="text-lg mr-3">{item.flag}</Text>
            <Text
              className={cn(
                "text-base",
                countryCode === item.code
                  ? "text-primary-700 font-medium"
                  : "text-neutral-800"
              )}
            >
              {item.name}
            </Text>
          </Pressable>
        )}
        showsVerticalScrollIndicator={false}
      />
    </View>
  )}
</View>

{/* Unidades de peso */}
<View className="mb-6">
  <Text className="text-sm font-medium text-neutral-700 mb-1.5">
    Unidades de peso
```

```
      </Text>
      <View className="flex-row gap-3">
        <ToggleOption
          label="kg"
          isSelected={unitWeight === "kg"}
          onPress={() => setField("unitWeight", "kg")}
        />
        <ToggleOption
          label="lb"
          isSelected={unitWeight === "lb"}
          onPress={() => setField("unitWeight", "lb")}
        />
      </View>
    </View>

    {/* Unidades de energía */}
    <View className="mb-6">
      <Text className="text-sm font-medium text-neutral-700 mb-1.5">
        Unidades de energía
      </Text>
      <View className="flex-row gap-3">
        <ToggleOption
          label="kcal"
          isSelected={unitEnergy === "kcal"}
          onPress={() => setField("unitEnergy", "kcal")}
        />
        <ToggleOption
          label="kJ"
          isSelected={unitEnergy === "kJ"}
          onPress={() => setField("unitEnergy", "kJ")}
        />
      </View>
    </View>

    {/* Zona horaria (detectada automáticamente) */}
    <View className="mb-6">
      <Text className="text-sm font-medium text-neutral-700 mb-1.5">
        Zona horaria
      </Text>
      <View className="bg-white border border-neutral-200 rounded-xl px-4 py-
        <Text className="text-base text-neutral-900">{timezone}</Text>
        <Text className="text-xs text-neutral-400 mt-1">
          Detectada automáticamente
        </Text>
      </View>
    </View>
  </ScrollView>
```

```
      {/* Botones */}
      <View className="flex-row gap-3 px-6">
        <Pressable
          onPress={handleBack}
          accessibilityLabel="Volver al paso anterior"
          accessibilityRole="button"
          className="flex-1 py-4 rounded-2xl items-center border border-neutral-2
        >
          <Text className="text-neutral-600 font-semibold text-base">Atrás</Text>
        </Pressable>
        <Pressable
          onPress={handleNext}
          accessibilityLabel="Continuar al siguiente paso"
          accessibilityRole="button"
          className="flex-[2] py-4 rounded-2xl items-center bg-primary-500 active
        >
          <Text className="text-white font-semibold text-base">Continuar</Text>
        </Pressable>
      </View>
    </View>
  );
}

// Toggle genérico de 2 opciones
function ToggleOption({
  label,
  isSelected,
  onPress,
}: {
  label: string;
  isSelected: boolean;
  onPress: () => void;
}) {
  return (
    <Pressable
      onPress={onPress}
      accessibilityLabel={`Unidad: ${label}`}
      accessibilityRole="button"
      accessibilityState={{ selected: isSelected }}
      className={cn(
        "flex-1 py-3.5 rounded-xl items-center border-2",
        isSelected
          ? "border-primary-500 bg-primary-50"
          : "border-neutral-200 bg-white"
      )}
    >
```

```
      <Text
        className={cn(
          "text-base font-semibold",
          isSelected ? "text-primary-700" : "text-neutral-600"
        )}
      >
        {label}
      </Text>
    </Pressable>
  );
}
```

---

src/app/(onboarding)/ready.tsx

```
// src/app/(onboarding)/ready.tsx
// Pantalla final — paso 7 del onboarding.
// Calcula TDEE con lógica pura local (sin red).
// Muestra objetivo calórico + macros de forma amigable.
// Botón "¡Empezar!" ejecuta submitOnboarding() y el guard redirige a (tabs).

import { useEffect, useMemo } from "react";
import { View, Text, Pressable, ActivityIndicator } from "react-native";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withTiming,
  withDelay,
  Easing,
} from "react-native-reanimated";
import { useAuthStore } from "@/stores/authStore";
import { useOnboardingStore } from "@/stores/onboardingStore";
import { calculateInitialTdee } from "@/features/tdee/algorithm";
import { submitOnboarding } from "@/features/onboarding/submitOnboarding";

export default function ReadyScreen() {
  const insets = useSafeAreaInsets();
  const userId = useAuthStore((s) => s.user?.id);
  const store = useOnboardingStore();

  // Calcular TDEE — lógica pura, sin llamada de red
  const result = useMemo(() => {
    if (
      !store.weightKg ||
```

```
    !store.heightCm ||
    !store.dateOfBirth ||
    !store.biologicalSex ||
    !store.goal ||
    !store.activityLevel
  ) {
    return null;
  }

  return calculateInitialTdee(
    {
      biological_sex: store.biologicalSex,
      date_of_birth: store.dateOfBirth,
      height_cm: store.heightCm,
      activity_level: store.activityLevel,
      goal: store.goal,
    },
    store.weightKg
  );
}, [
  store.weightKg,
  store.heightCm,
  store.dateOfBirth,
  store.biologicalSex,
  store.goal,
  store.activityLevel,
]);

// Animaciones de entrada escalonadas
const titleOpacity = useSharedValue(0);
const numberScale = useSharedValue(0.8);
const numberOpacity = useSharedValue(0);
const macrosOpacity = useSharedValue(0);
const buttonOpacity = useSharedValue(0);

useEffect(() => {
  titleOpacity.value = withTiming(1, { duration: 400, easing: Easing.out(Easing
  numberOpacity.value = withDelay(200, withTiming(1, { duration: 500 }));
  numberScale.value = withDelay(200, withTiming(1, { duration: 500, easing: Eas
  macrosOpacity.value = withDelay(500, withTiming(1, { duration: 400 }));
  buttonOpacity.value = withDelay(800, withTiming(1, { duration: 400 }));
}, []);

const titleStyle = useAnimatedStyle(() => ({ opacity: titleOpacity.value }));
const numberStyle = useAnimatedStyle(() => ({
  opacity: numberOpacity.value,
  transform: [{ scale: numberScale.value }],
```

```
}));
const macrosStyle = useAnimatedStyle(() => ({ opacity: macrosOpacity.value }));
const buttonStyle = useAnimatedStyle(() => ({ opacity: buttonOpacity.value }));

// Texto del objetivo legible
const goalText =
  store.goal === "lose_weight"
    ? "Perder peso"
    : store.goal === "gain_muscle"
      ? "Ganar músculo"
      : "Mantener peso";

// Handler de submit
const handleStart = async () => {
  if (!userId || store.isSubmitting) return;
  store.setSubmitting(true);
  store.setSubmitError(null);

  try {
    await submitOnboarding(store, userId);
    // Stack.Protected detecta hasCompletedOnboarding = true
    // y redirige automáticamente a (tabs). No hacer router.replace().
  } catch (err: any) {
    store.setSubmitError(
      err.message || "Hubo un error al guardar. Inténtalo de nuevo."
    );
    store.setSubmitting(false);
  }
};

if (!result) {
  return (
    <View className="flex-1 bg-neutral-50 items-center justify-center">
      <Text className="text-neutral-500">Calculando tu plan...</Text>
    </View>
  );
}

const { calorie_goal } = result;

return (
  <View
    className="flex-1 bg-neutral-50 px-6 justify-between"
    style={{ paddingTop: 24, paddingBottom: insets.bottom + 24 }}
  >
    {/* Contenido principal */}
    <View className="flex-1 justify-center items-center">
```

```
{/* Título */}
<Animated.View style={titleStyle}>
  <Text className="font-display text-2xl text-neutral-900 text-center mb-
    ¡Tu plan está listo! 🎉
  </Text>
  <Text className="text-neutral-500 text-center mb-8">
    Objetivo: {goalText}
  </Text>
</Animated.View>

{/* Número grande del objetivo calórico */}
<Animated.View style={numberStyle} className="items-center mb-8">
  <Text className="text-neutral-500 text-sm mb-1">
    Tu objetivo diario
  </Text>
  <Text className="font-display text-6xl text-primary-600">
    {calorie_goal.goal_kcal.toLocaleString("es")}
  </Text>
  <Text className="text-neutral-400 text-lg">
    {store.unitEnergy}
  </Text>
</Animated.View>

{/* Desglose de macros */}
<Animated.View style={macrosStyle} className="flex-row gap-4 mb-6">
  <MacroBox
    label="Proteína"
    value={calorie_goal.protein_g}
    unit="g"
    color="text-secondary-600"
    bg="bg-secondary-50"
  />
  <MacroBox
    label="Carbos"
    value={calorie_goal.carbs_g}
    unit="g"
    color="text-primary-600"
    bg="bg-primary-50"
  />
  <MacroBox
    label="Grasa"
    value={calorie_goal.fat_g}
    unit="g"
    color="text-warm-600"
    bg="bg-warm-50"
  />
</Animated.View>
```

```
      {/* Etiqueta de confianza */}
      <Animated.View style={macrosStyle}>
        <Text className="text-neutral-400 text-sm text-center">
          {calorie_goal.confidence_label} — irá mejorando cada semana
        </Text>
      </Animated.View>
    </View>

    {/* Error de submit */}
    {store.submitError && (
      <View className="bg-warm-50 border border-warm-200 rounded-xl p-3 mb-3">
        <Text className="text-warm-700 text-sm text-center">
          {store.submitError}
        </Text>
      </View>
    )}

    {/* Botón de empezar */}
    <Animated.View style={buttonStyle}>
      <Pressable
        onPress={handleStart}
        disabled={store.isSubmitting}
        accessibilityLabel="Empezar a usar Nutria"
        accessibilityRole="button"
        className={cn(
          "py-4 rounded-2xl items-center",
          store.isSubmitting
            ? "bg-primary-300"
            : "bg-primary-500 active:bg-primary-600"
        )}
      >
        {store.isSubmitting ? (
          <ActivityIndicator color="white" />
        ) : (
          <Text className="text-white font-semibold text-lg">
            ¡Empezar con Nutria!
          </Text>
        )}
      </Pressable>

      <Text className="text-neutral-400 text-xs text-center mt-3">
        Podrás cambiar tu objetivo y preferencias cuando quieras
      </Text>
    </Animated.View>
  </View>
);
```

```
}

// Subcomponente: caja de macro individual
function MacroBox({
  label,
  value,
  unit,
  color,
  bg,
}: {
  label: string;
  value: number;
  unit: string;
  color: string;
  bg: string;
}) {
  return (
    <View className={cn("flex-1 items-center py-4 rounded-xl", bg)}>
      <Text className={cn("font-display text-2xl font-bold", color)}>
        {value}
        <Text className="text-sm font-normal">{unit}</Text>
      </Text>
      <Text className="text-neutral-500 text-xs mt-1">{label}</Text>
    </View>
  );
}
```

---

src/app/(onboarding)/tca-screening.tsx

```
// src/app/(onboarding)/tca-screening.tsx
// Screening suave de TCA — paso 5 del onboarding.
// TOTALMENTE OPCIONAL. El botón "Continuar" siempre está activo.
// Si selecciona "Complicada", se muestra un mensaje cálido de apoyo.
// Si no selecciona nada y avanza, se guarda null.

import { View, Text, Pressable, ScrollView } from "react-native";
import { useRouter } from "expo-router";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withTiming,
  withSpring,
  Easing,
```

```
} from "react-native-reanimated";
import { cn } from "@/lib/cn";
import { useOnboardingStore } from "@/stores/onboardingStore";
import type { TcaScreeningAnswer } from "@/stores/onboardingStore";

const OPTIONS: Array<{
  key: TcaScreeningAnswer;
  emoji: string;
  text: string;
}> = [
  {
    key: "very_positive",
    emoji: "😊",
    text: "Muy positiva — disfruto comer sin preocupaciones",
  },
  {
    key: "positive",
    emoji: "🙂",
    text: "Positiva — a veces me preocupa un poco",
  },
  {
    key: "neutral",
    emoji: "😐",
    text: "Neutral — no lo pienso mucho",
  },
  {
    key: "complicated",
    emoji: "💛",
    text: "Complicada — tengo o he tenido dificultades",
  },
  {
    key: "prefer_not_to_say",
    emoji: "🤐",
    text: "Prefiero no responder",
  },
];

function OptionCard({
  option,
  isSelected,
  onPress,
}: {
  option: (typeof OPTIONS)[0];
  isSelected: boolean;
  onPress: () => void;
}) {
  const scale = useSharedValue(1);
```

```
  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  const handlePress = () => {
    scale.value = withSpring(0.96, { damping: 15, stiffness: 300 }, () => {
      scale.value = withSpring(1, { damping: 12, stiffness: 200 });
    });
    onPress();
  };

  return (
    <Animated.View style={animatedStyle}>
      <Pressable
        onPress={handlePress}
        accessibilityLabel={option.text}
        accessibilityRole="button"
        accessibilityState={{ selected: isSelected }}
        className={cn(
          "flex-row items-center px-4 py-3.5 rounded-xl mb-2.5 border-2",
          isSelected
            ? "border-primary-500 bg-primary-50"
            : "border-neutral-200 bg-white"
        )}
      >
        <Text className="text-xl mr-3">{option.emoji}</Text>
        <Text
          className={cn(
            "flex-1 text-base",
            isSelected ? "text-primary-700 font-medium" : "text-neutral-700"
          )}
        >
          {option.text}
        </Text>
      </Pressable>
    </Animated.View>
  );
}

export default function TcaScreeningScreen() {
  const router = useRouter();
  const insets = useSafeAreaInsets();
  const { tcaScreening, setField, nextStep, prevStep } = useOnboardingStore();

  // Mensaje de apoyo visible solo si selecciona "complicated"
  const supportOpacity = useSharedValue(0);
```

```
const supportHeight = useSharedValue(0);

const supportStyle = useAnimatedStyle(() => ({
  opacity: supportOpacity.value,
  maxHeight: supportHeight.value,
  overflow: "hidden" as const,
}));

const handleSelect = (key: TcaScreeningAnswer) => {
  setField("tcaScreening", key);
  if (key === "complicated") {
    supportOpacity.value = withTiming(1, { duration: 400, easing: Easing.out(Ea
    supportHeight.value = withTiming(160, { duration: 400, easing: Easing.out(E
  } else {
    supportOpacity.value = withTiming(0, { duration: 200 });
    supportHeight.value = withTiming(0, { duration: 200 });
  }
};

const handleNext = () => {
  // No requiere selección — guardar null si no eligió nada
  nextStep();
  router.push("/(onboarding)/preferences");
};

const handleBack = () => {
  prevStep();
  router.back();
};

return (
  <View
    className="flex-1 bg-neutral-50"
    style={{ paddingBottom: insets.bottom + 24 }}
  >
    <ScrollView
      className="flex-1 px-6"
      contentContainerClassName="pt-4 pb-4"
      showsVerticalScrollIndicator={false}
    >
      <Text className="font-display text-2xl text-neutral-900 mb-2">
        Una pregunta personal
      </Text>
      <Text className="text-neutral-500 mb-1">
        ¿Cómo describirías tu relación con la comida?
      </Text>
      <Text className="text-neutral-400 text-sm mb-6">
```

```
          Es totalmente opcional — puedes pasar directamente.
        </Text>

        {OPTIONS.map((option) => (
          <OptionCard
            key={option.key}
            option={option}
            isSelected={tcaScreening === option.key}
            onPress={() => handleSelect(option.key)}
          />
        ))}

        {/* Mensaje de apoyo para "complicada" */}
        <Animated.View style={supportStyle}>
          <View className="bg-warm-50 border border-warm-200 rounded-xl p-4 mt-2"
            <Text className="text-warm-700 text-base leading-6">
              💛 Gracias por compartirlo. Nutria está diseñada para ser una
              herramienta de apoyo, nunca de presión. Puedes usar la app a tu
              ritmo, y siempre puedes ocultar las calorías si lo prefieres.
            </Text>
          </View>
        </Animated.View>
      </ScrollView>

      {/* Botones — "Continuar" SIEMPRE activo */}
      <View className="flex-row gap-3 px-6">
        <Pressable
          onPress={handleBack}
          accessibilityLabel="Volver al paso anterior"
          accessibilityRole="button"
          className="flex-1 py-4 rounded-2xl items-center border border-neutral-2
        >
          <Text className="text-neutral-600 font-semibold text-base">Atrás</Text>
        </Pressable>
        <Pressable
          onPress={handleNext}
          accessibilityLabel="Continuar al siguiente paso"
          accessibilityRole="button"
          className="flex-[2] py-4 rounded-2xl items-center bg-primary-500 active
        >
          <Text className="text-white font-semibold text-base">Continuar</Text>
        </Pressable>
      </View>
    </View>
  );
}
```

src/app/(onboarding)/welcome.tsx

```
// src/app/(onboarding)/welcome.tsx
// Pantalla de bienvenida — paso 1 del onboarding.
// Sin progress bar, sin botón atrás.
// Animación de entrada: fade + slide up.

import { useEffect } from "react";
import { View, Text, Pressable } from "react-native";
import { useRouter } from "expo-router";
import { Image } from "expo-image";
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withTiming,
  Easing,
} from "react-native-reanimated";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { useAuthStore } from "@/stores/authStore";
import { useOnboardingStore } from "@/stores/onboardingStore";

export default function WelcomeScreen() {
  const router = useRouter();
  const insets = useSafeAreaInsets();
  const user = useAuthStore((s) => s.user);
  const { setField, nextStep } = useOnboardingStore();

  // Nombre del usuario (de Supabase Auth metadata o email)
  const userName =
    user?.user_metadata?.display_name ||
    user?.email?.split("@")[0] ||
    "";

  // Animación de entrada
  const opacity = useSharedValue(0);
  const translateY = useSharedValue(30);

  useEffect(() => {
    opacity.value = withTiming(1, {
      duration: 600,
      easing: Easing.out(Easing.cubic),
    });
    translateY.value = withTiming(0, {
      duration: 600,
      easing: Easing.out(Easing.cubic),
```

```
  });
}, []);

const animatedStyle = useAnimatedStyle(() => ({
  opacity: opacity.value,
  transform: [{ translateY: translateY.value }],
}));

const handleStart = () => {
  // Pre-cargar el nombre si existe
  if (userName) {
    setField("displayName", userName);
  }
  nextStep();
  router.push("/(onboarding)/body-profile");
};

return (
  <View
    className="flex-1 bg-neutral-50 px-6 justify-between"
    style={{ paddingTop: insets.top + 40, paddingBottom: insets.bottom + 24 }}
  >
    <Animated.View style={animatedStyle} className="items-center flex-1 justify
      {/* Ilustración hero */}
      <Image
        source={require("@/../../assets/images/onboarding-hero.png")}
        style={{ width: 240, height: 240 }}
        contentFit="contain"
        accessibilityLabel="Ilustración de bienvenida a Nutria"
      />

      {/* Saludo */}
      <Text className="font-display text-3xl text-neutral-900 text-center mt-8
        {userName ? `Hola, ${userName} 👋` : "¡Bienvenido a Nutria! 👋"}
      </Text>

      {/* Propuesta de valor */}
      <Text className="text-base text-neutral-500 text-center leading-6 px-4">
        Tu compañero de nutrición inteligente.{"\n"}
        Aprende de ti para ayudarte a comer mejor, sin presiones.
      </Text>
    </Animated.View>

    {/* Botón de empezar */}
    <Animated.View style={animatedStyle}>
      <Pressable
        onPress={handleStart}
```

```
          accessibilityLabel="Empezar configuración"
          accessibilityRole="button"
          className="bg-primary-500 py-4 rounded-2xl items-center active:bg-prima
        >
          <Text className="text-white font-semibold text-base">Empezar</Text>
        </Pressable>
      </Animated.View>
    </View>
  );
}
```

---

src/components/onboarding/ActivitySelector.tsx

```
// src/components/onboarding/ActivitySelector.tsx
// Selector visual de nivel de actividad: 5 cards verticales.

import { View, Text, Pressable } from "react-native";
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from "react-native-reanimated";
import { cn } from "@/lib/cn";
import type { ActivityLevel } from "@/stores/onboardingStore";

interface ActivitySelectorProps {
  value: ActivityLevel | null;
  onChange: (level: ActivityLevel) => void;
}

const LEVELS: Array<{
  key: ActivityLevel;
  emoji: string;
  title: string;
  description: string;
}> = [
  {
    key: "sedentary",
    emoji: "🪑",
    title: "Sedentario",
    description: "Trabajo de oficina, poco ejercicio",
  },
  {
    key: "lightly_active",
```

```
    emoji: "🚶",
    title: "Ligeramente activo",
    description: "Ejercicio ligero 1-3 días/semana",
  },
  {
    key: "moderately_active",
    emoji: "🏃",
    title: "Moderadamente activo",
    description: "Ejercicio moderado 3-5 días/semana",
  },
  {
    key: "very_active",
    emoji: "🏋️",
    title: "Muy activo",
    description: "Ejercicio intenso 6-7 días/semana",
  },
  {
    key: "extra_active",
    emoji: "⚡",
    title: "Extra activo",
    description: "Trabajo físico o atleta profesional",
  },
];

function ActivityCard({
  level,
  isSelected,
  onPress,
}: {
  level: (typeof LEVELS)[0];
  isSelected: boolean;
  onPress: () => void;
}) {
  const scale = useSharedValue(1);

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  const handlePress = () => {
    scale.value = withSpring(0.96, { damping: 15, stiffness: 300 }, () => {
      scale.value = withSpring(1, { damping: 12, stiffness: 200 });
    });
    onPress();
  };

  return (
```

```
    <Animated.View style={animatedStyle}>
      <Pressable
        onPress={handlePress}
        accessibilityLabel={`${level.title}: ${level.description}`}
        accessibilityRole="button"
        accessibilityState={{ selected: isSelected }}
        className={cn(
          "flex-row items-center px-4 py-3.5 rounded-xl mb-2.5 border-2",
          isSelected
            ? "border-primary-500 bg-primary-50"
            : "border-neutral-200 bg-white"
        )}
      >
        <Text className="text-2xl mr-3">{level.emoji}</Text>
        <View className="flex-1">
          <Text
            className={cn(
              "text-base font-semibold",
              isSelected ? "text-primary-700" : "text-neutral-800"
            )}
          >
            {level.title}
          </Text>
          <Text
            className={cn(
              "text-sm",
              isSelected ? "text-primary-600" : "text-neutral-500"
            )}
          >
            {level.description}
          </Text>
        </View>
      </Pressable>
    </Animated.View>
  );
}

export function ActivitySelector({ value, onChange }: ActivitySelectorProps) {
  return (
    <View>
      {LEVELS.map((level) => (
        <ActivityCard
          key={level.key}
          level={level}
          isSelected={value === level.key}
          onPress={() => onChange(level.key)}
        />
```

```
      ))}
    </View>
  );
}
```

---

src/components/onboarding/GoalSelector.tsx

```
// src/components/onboarding/GoalSelector.tsx
// Selector visual de objetivo: 3 cards grandes, una selección.

import { View, Text, Pressable } from "react-native";
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from "react-native-reanimated";
import { cn } from "@/lib/cn";
import type { UserGoal } from "@/stores/onboardingStore";

interface GoalSelectorProps {
  value: UserGoal | null;
  onChange: (goal: UserGoal) => void;
}

const GOALS: Array<{
  key: UserGoal;
  emoji: string;
  title: string;
  description: string;
}> = [
  {
    key: "lose_weight",
    emoji: "🎯",
    title: "Perder peso",
    description: "Un plan cómodo y sostenible, sin pasar hambre",
  },
  {
    key: "maintain",
    emoji: "⚖️",
    title: "Mantener",
    description: "Come lo que gastas, con consciencia y equilibrio",
  },
  {
    key: "gain_muscle",
```

```
    emoji: "💪",
    title: "Ganar músculo",
    description: "Superávit controlado para construir masa muscular",
  },
];

function GoalCard({
  goal,
  isSelected,
  onPress,
}: {
  goal: (typeof GOALS)[0];
  isSelected: boolean;
  onPress: () => void;
}) {
  // Animación de escala al seleccionar
  const scale = useSharedValue(1);

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  const handlePress = () => {
    scale.value = withSpring(0.95, { damping: 15, stiffness: 300 }, () => {
      scale.value = withSpring(1, { damping: 12, stiffness: 200 });
    });
    onPress();
  };

  return (
    <Animated.View style={animatedStyle}>
      <Pressable
        onPress={handlePress}
        accessibilityLabel={`${goal.title}: ${goal.description}`}
        accessibilityRole="button"
        accessibilityState={{ selected: isSelected }}
        className={cn(
          "flex-row items-center p-5 rounded-2xl mb-3 border-2",
          isSelected
            ? "border-primary-500 bg-primary-50"
            : "border-neutral-200 bg-white"
        )}
      >
        <Text className="text-3xl mr-4">{goal.emoji}</Text>
        <View className="flex-1">
          <Text
            className={cn(
```

```
              "text-lg font-semibold mb-1",
              isSelected ? "text-primary-700" : "text-neutral-800"
            )}
          >
            {goal.title}
          </Text>
          <Text
            className={cn(
              "text-sm",
              isSelected ? "text-primary-600" : "text-neutral-500"
            )}
          >
            {goal.description}
          </Text>
        </View>
      </Pressable>
    </Animated.View>
  );
}

export function GoalSelector({ value, onChange }: GoalSelectorProps) {
  return (
    <View>
      {GOALS.map((goal) => (
        <GoalCard
          key={goal.key}
          goal={goal}
          isSelected={value === goal.key}
          onPress={() => onChange(goal.key)}
        />
      ))}
    </View>
  );
}
```

---

src/components/onboarding/StepProgressBar.tsx

```
// src/components/onboarding/StepProgressBar.tsx
// Barra de progreso animada para el onboarding.
// Se anima suavemente al cambiar de paso con Reanimated v4.

import { useEffect } from "react";
import { View } from "react-native";
import Animated, {
```

```
  useSharedValue,
  useAnimatedStyle,
  withTiming,
  Easing,
} from "react-native-reanimated";
import { useSafeAreaInsets } from "react-native-safe-area-context";

interface StepProgressBarProps {
  currentStep: number; // 1-7
  totalSteps: number; // 7
}

export function StepProgressBar({
  currentStep,
  totalSteps,
}: StepProgressBarProps) {
  const insets = useSafeAreaInsets();

  // Paso 1 (welcome) no muestra la barra, así que el progreso efectivo
  // va de paso 2 (0%) a paso 7 (100%)
  const effectiveSteps = totalSteps - 1; // 6 pasos con barra
  const effectiveCurrent = currentStep - 1; // paso 2 = 1, paso 7 = 6
  const targetProgress = Math.min(effectiveCurrent / effectiveSteps, 1);

  const progress = useSharedValue(0);

  useEffect(() => {
    progress.value = withTiming(targetProgress, {
      duration: 350,
      easing: Easing.out(Easing.cubic),
    });
  }, [targetProgress]);

  const animatedStyle = useAnimatedStyle(() => ({
    width: `${progress.value * 100}%`,
  }));

  return (
    <View
      className="px-6 pb-2"
      style={{ paddingTop: insets.top + 8 }}
    >
      {/* Fondo de la barra */}
      <View className="h-1 w-full rounded-full bg-neutral-200 overflow-hidden">
        {/* Relleno animado */}
        <Animated.View
          className="h-full rounded-full bg-primary-500"
```

```
          style={animatedStyle}
        />
      </View>
    </View>
  );
}
```

---

src/features/onboarding/submitOnboarding.ts

```
// src/features/onboarding/submitOnboarding.ts
// Escribe TODOS los datos del onboarding a Supabase en un solo flujo.
// Se ejecuta al final del onboarding (paso 7, ready.tsx).
// Si falla cualquier paso, propaga el error sin dejar datos parciales.

import { supabase } from "@/lib/supabase";
import { useAuthStore } from "@/stores/authStore";
import { calculateInitialTdee } from "@/features/tdee/algorithm";
import type { OnboardingState } from "@/stores/onboardingStore";

export async function submitOnboarding(
  data: OnboardingState,
  userId: string
): Promise<void> {
  // Validar que tenemos todos los datos necesarios
  if (
    !data.weightKg ||
    !data.heightCm ||
    !data.dateOfBirth ||
    !data.biologicalSex ||
    !data.goal ||
    !data.activityLevel
  ) {
    throw new Error("Faltan datos obligatorios del onboarding");
  }

  // 1. El peso ya está en kg (la conversión se hizo en body-profile.tsx)
  const weightKg = data.weightKg;
  const heightCm = data.heightCm;

  // 2. Calcular TDEE inicial
  const { bmr, tdee, calorie_goal } = calculateInitialTdee(
    {
      biological_sex: data.biologicalSex,
      date_of_birth: data.dateOfBirth,
```

```
    height_cm: heightCm,
    activity_level: data.activityLevel,
    goal: data.goal,
  },
  weightKg
);

// 3. Insertar peso inicial en weight_entries
const { error: weightError } = await supabase
  .from("weight_entries")
  .insert({
    user_id: userId,
    weight_kg: weightKg,
  });

if (weightError) {
  throw new Error(`Error guardando peso inicial: ${weightError.message}`);
}

// 4. Upsert en user_profiles (upsert por si hay re-intento tras error)
const { error: profileError } = await supabase
  .from("user_profiles")
  .upsert({
    id: userId,
    display_name: data.displayName || null,
    date_of_birth: data.dateOfBirth,
    biological_sex: data.biologicalSex,
    height_cm: heightCm,
    goal: data.goal,
    activity_level: data.activityLevel,
    tca_screening: data.tcaScreening, // puede ser null
    unit_weight: data.unitWeight,
    unit_energy: data.unitEnergy,
    country_code: data.countryCode,
    timezone: data.timezone,
    onboarding_completed: true,
  });

if (profileError) {
  throw new Error(`Error guardando perfil: ${profileError.message}`);
}

// 5. Insertar estado TDEE inicial (upsert por idempotencia)
const { error: tdeeError } = await supabase
  .from("user_tdee_state")
  .upsert({
    user_id: userId,
```

```
      current_tdee_kcal: tdee,
      current_bmr_kcal: bmr,
      initial_tdee_kcal: tdee,
      confidence_level: 0.3,
      weeks_of_data: 0,
    });

  if (tdeeError) {
    throw new Error(`Error guardando TDEE inicial: ${tdeeError.message}`);
  }

  // 6. Actualizar authStore — esto dispara el guard de Stack.Protected
  //    que redirige automáticamente a (tabs)
  useAuthStore.getState().setOnboardingCompleted(true);

  // No hacer router.replace() — Stack.Protected lo maneja
}
```

---

src/stores/onboardingStore.ts

```
// src/stores/onboardingStore.ts
// Store Zustand dedicado al flujo de onboarding.
// Vive solo durante el onboarding — se resetea al completar.
// NO contamina authStore ni uiStore.

import { create } from "zustand";

export type BiologicalSex = "male" | "female";
export type UserGoal = "lose_weight" | "maintain" | "gain_muscle";
export type ActivityLevel =
  | "sedentary"
  | "lightly_active"
  | "moderately_active"
  | "very_active"
  | "extra_active";
export type TcaScreeningAnswer =
  | "very_positive"
  | "positive"
  | "neutral"
  | "complicated"
  | "prefer_not_to_say";

export interface OnboardingState {
  // Datos recopilados paso a paso
```

```
  displayName: string;
  weightKg: number | null;
  heightCm: number | null;
  dateOfBirth: string | null;
  biologicalSex: BiologicalSex | null;
  goal: UserGoal | null;
  activityLevel: ActivityLevel | null;
  tcaScreening: TcaScreeningAnswer | null;
  countryCode: string;
  unitWeight: "kg" | "lb";
  unitEnergy: "kcal" | "kJ";
  timezone: string;

  // Control del flujo
  currentStep: number;
  isSubmitting: boolean;
  submitError: string | null;

  // Acciones
  setField: <K extends keyof OnboardingState>(
    key: K,
    value: OnboardingState[K]
  ) => void;
  nextStep: () => void;
  prevStep: () => void;
  setSubmitting: (v: boolean) => void;
  setSubmitError: (msg: string | null) => void;
  reset: () => void;
}

const initialState = {
  displayName: "",
  weightKg: null,
  heightCm: null,
  dateOfBirth: null,
  biologicalSex: null,
  goal: null,
  activityLevel: null,
  tcaScreening: null,
  countryCode: "ES",
  unitWeight: "kg" as const,
  unitEnergy: "kcal" as const,
  timezone: Intl.DateTimeFormat().resolvedOptions().timeZone || "Europe/Madrid",
  currentStep: 1,
  isSubmitting: false,
  submitError: null,
};
```

```
export const useOnboardingStore = create<OnboardingState>((set) => ({
  ...initialState,

  setField: (key, value) => set({ [key]: value } as Partial<OnboardingState>),

  nextStep: () =>
    set((s) => ({ currentStep: Math.min(s.currentStep + 1, 7) })),

  prevStep: () =>
    set((s) => ({ currentStep: Math.max(s.currentStep - 1, 1) })),

  setSubmitting: (v) => set({ isSubmitting: v }),

  setSubmitError: (msg) => set({ submitError: msg }),

  reset: () => set(initialState),
}));
```